

TRƯỜNG ĐẠI HỌC ĐẠI NAM
KHOA CÔNG NGHỆ THÔNG TIN
BÁO CÁO BÀI TẬP LỚN

Học phần: Dịch vụ kết nối và Công nghệ nền tảng (FIT4110)

Đề tài: Xây dựng service AI Vision (A4) trong Smart Campus Operations Platform

Mã nhóm: Nhóm 4 Lớp: CNTT 17-08

Sản phẩm (Product): Smart Campus Operations Platform (Product A)

Service phụ trách: A4 - AI Vision Service

Giảng viên hướng dẫn: ThS. Lê Thái Bảo

Năm học: 2025 - 2026

Danh sách thành viên:

STT	Họ và tên	Mã sinh viên	Vai trò
1	Nguyễn Thanh Tùng	1771020729	Nhóm trưởng
2	Nguyễn Đức Anh	1771020550	Thành viên
3	Lê Văn Vượng	1771020771	Thành viên

I. Giới thiệu

1.1. Bối cảnh

Smart Campus Operations Platform là hệ thống quản lý khuôn viên trường học thông minh gồm 7 microservice giao tiếp qua REST API và MQTT. Trong kiến trúc hiện tại, AI Vision Service (A4) là dịch vụ xử lý thị giác máy tính, nhận ảnh từ Camera Stream Service (A2) và cung cấp kết quả nhận diện cho Camera Stream/Core Business (A6). Service A4 giúp hệ thống chuyển dữ liệu ảnh thô thành thông tin nghiệp vụ như: có người trong ảnh hay không, mức rủi ro của khung hình, và kết quả đối chiếu khuôn mặt khi Core Business yêu cầu.

Theo repo nhánh deploy-lan, hệ thống hiện có đủ 7 service: IoT Ingestion, Camera Stream, Access Gate, AI Vision, Analytics, Core Business và Notification. A4 chạy ở port 8004, container smart-campus-ai-vision. Đặc biệt, toàn bộ mã nguồn của AI Vision Service được tổ chức chặt chẽ theo chuẩn cấu trúc microservices tại thư mục services/ai_vision bao gồm: thư mục src/ chứa lõi logic (entry point tại main.py), thư mục contracts/ chứa đặc tả API, thư mục postman/ quản lý kịch bản kiểm thử, và docs/ chứa các tài liệu đánh giá (evaluation.md), trình bày (presentation.md).

1.2. Mục tiêu của service

AI Vision Service sinh ra để phân tích ảnh/snapshot từ camera, phát hiện khuôn mặt hoặc người xuất hiện trong khung hình, đánh giá mức rủi ro ban đầu và hỗ trợ Core Business trong nghiệp vụ xác thực danh tính qua face-match.

1.3. Phạm vi

Service có làm:

- 1. Nhận ảnh qua REST POST /detect từ Camera Stream Service (A2), hỗ trợ image_url hoặc image_base64.
- 2. Giải mã ảnh bằng OpenCV, chạy Haar Cascade Face Detector để tìm khuôn mặt/người trong khung hình.
- 3. Trả về detection_id, danh sách detections, bbox, confidence, unknown_person và risk_level.

4. Cung cấp REST POST /vision/face-match cho Core Business Service (A6) khi cần đối chiếu khuôn mặt.

5. Lưu tạm kết quả detection trong bộ nhớ để truy vấn lại qua GET /vision/detections/{detection_id}.

6. Cung cấp endpoint /vision/models/info để mô tả model đang dùng và /health để kiểm tra service sống.

- Service không làm:

1. Không trực tiếp publish MQTT event; Camera Stream hoặc Core Business sẽ sử dụng kết quả A4 để phát event/cảnh báo.

2. Không lưu trữ ảnh và kết quả nhận diện lâu dài trong database; hiện tại dùng in-memory storage.

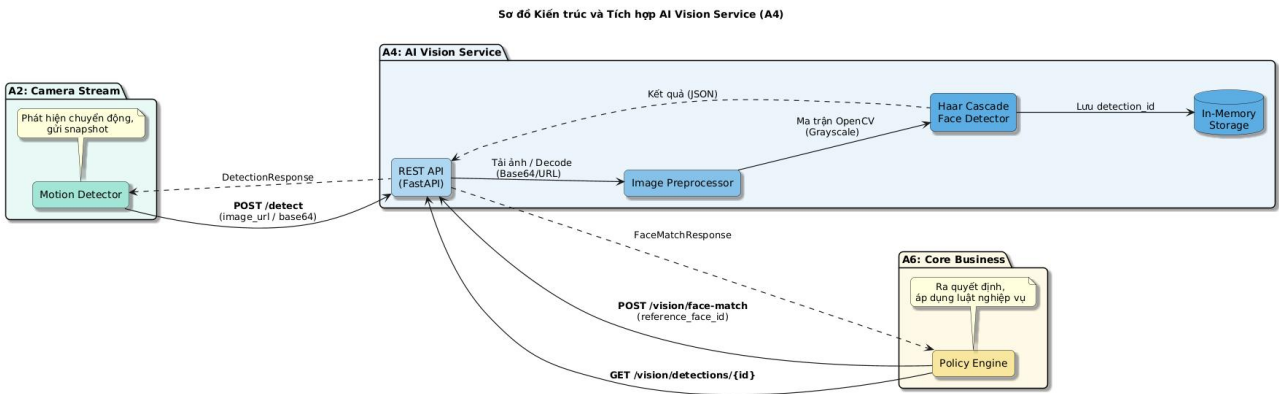
3. Không tự quyết định mở cửa hay gửi cảnh báo cuối cùng; quyết định nghiệp vụ thuộc Core Business.

4. Không thay thế hệ thống nhận diện khuôn mặt chính thức có dữ liệu huấn luyện đầy đủ; hiện tại triển khai theo hướng demo/lab bằng Haar Cascade và logic mô phỏng face-match.

II. Phân tích nghiệp vụ (phần trọng tâm)

AI Vision Service đóng vai trò là “bộ phân tích hình ảnh” của Smart Campus. Dịch vụ nhận ảnh từ camera, biến đổi dữ liệu ảnh thành kết quả có cấu trúc để các service khác có thể xử lý nghiệp vụ: Camera Stream dùng kết quả phát hiện người/chuyển động để tạo sự kiện camera, còn Core Business dùng kết quả face-match để xác thực danh tính hoặc đánh giá rủi ro an ninh.

2.2.1. Sơ đồ vị trí của AI Vision Service (A4) trong luồng tích hợp A2 - A4 - A6:



Hình 2.1. Sơ đồ vị trí của AI Vision Service (A4) trong luồng tích hợp A2 - A4 - A6.

2.2.2. Mô tả vai trò tích hợp với các service đối tác:

AI Vision Service (A4) nằm giữa Camera Stream Service (A2) và Core Business Service (A6). Khi A2 phát hiện chuyển động, nó gửi ảnh sang A4 để xác định xem có người hay vật thể lạ xuất hiện. Kết quả này được A2 chuyển hóa thành event gửi lên MQTT broker. Khi sinh viên quét thẻ hoặc có sự kiện nghi ngờ tại gate, A6 gọi trực tiếp API của A4 để đối chiếu khuôn mặt nhằm quyết định cho phép truy cập.

Quan hệ	Nhóm đối tác	Mục đích	Cơ chế (REST/MQTT)
Nhận từ	Camera Stream Service (A2)	Nhận snapshot khi A2 phát hiện motion, phân tích ảnh để tìm người/khuôn mặt.	REST POST /detect
Gửi tới	Camera Stream Service (A2)	Trả DetectionResponse để A2 đóng gói camera	REST Response

Quan hệ	Nhóm đối tác	Mục đích	Cơ chế (REST/MQTT)
		event hoặc hiển thị kết quả.	
Nhận từ	Core Business Service (A6)	Nhận yêu cầu đối chiếu khuôn mặt với reference_face_id khi cần xác thực danh tính.	REST POST /vision/face-match
Gửi tới	Core Business Service (A6)	Trả FaceMatchResponse gồm matched, confidence, model_version để A6 quyết định nghiệp vụ.	REST Response

2.3. Đặc tả dữ liệu đầu vào

Nguồn vào 1: REST POST /detect - Camera Stream gọi khi phát hiện motion.

Nguồn vào 2: REST POST /vision/face-match - Core Business gọi khi cần xác thực danh tính.

Nguồn vào 3: REST GET /vision/detections/{detection_id} - truy vấn lại kết quả detection đã xử lý.

Schema đầu vào:

Request body mẫu cho POST /detect (từ A2 gửi sang):

```
{
  "request_id": "cam-req-001",
  "camera_id": "cam-gate-a",
  "image_url": "https://example.com/frame.jpg",
  "image_base64": null,
  "timestamp": "2026-06-27T06:00:10Z",
  "motion_score": 0.85
}
```

Request body mẫu cho POST /vision/face-match (từ A6 gửi sang):

```
{
  "camera_id": "cam-gate-a",
  "image_url": null,
  "image_base64": "<base64-image>",
}
```

```
"reference_face_id": "SV1771020729"  
}
```

Request mẫu cho GET /vision/detections/{detection_id} (truy vấn nội bộ):

GET /vision/detections/DET-A1B2C3D4

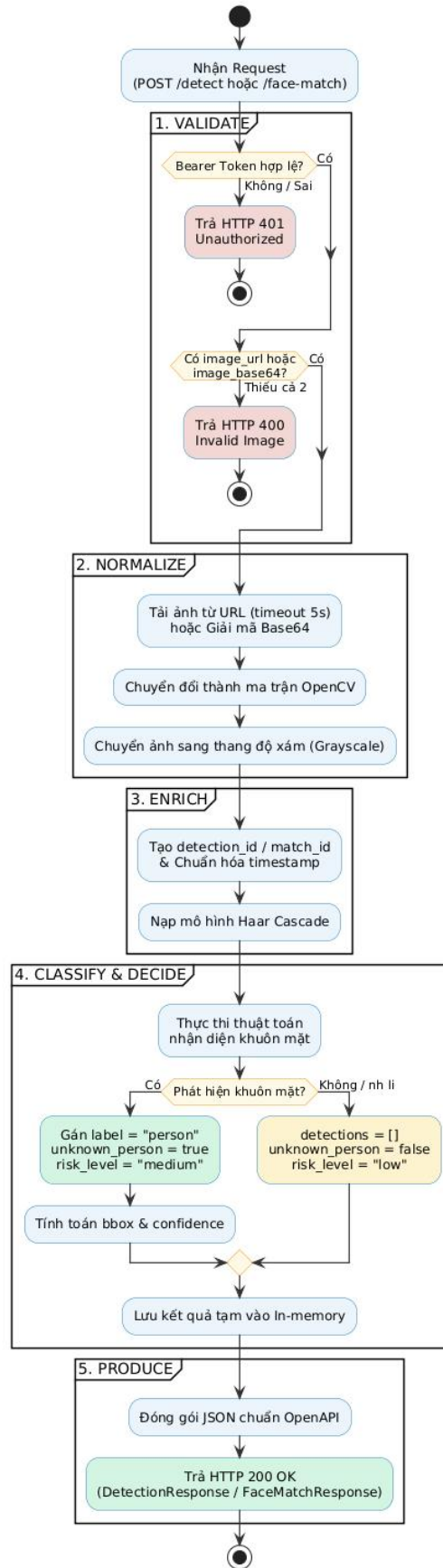
Authorization: Bearer <AUTH_TOKEN>

2.4. Logic xử lý nghiệp vụ

Quy trình xử lý ảnh của AI Vision Service bao gồm các bước từ xác thực token, tải ảnh từ URL hoặc giải mã chuỗi base64, chuyển đổi mảng byte thành định dạng ma trận OpenCV, nạp mô hình Haar Cascade để phát hiện khuôn mặt, phân loại mức độ rủi ro dựa trên sự tồn tại của khuôn mặt, và đóng gói dữ liệu trả về.

Bước	Service làm gì ở bước này
1. VALIDATE	Kiểm tra Authorization Bearer Token với toàn bộ API nghiệp vụ. Riêng /health mở để phục vụ healthcheck. Với /detect, bắt buộc có ít nhất một trong hai nguồn ảnh: image_url hoặc image_base64; nếu thiếu trả 400 Invalid image.
2. NORMALIZE	Chuẩn hóa timestamp, tạo detection_id/match_id dạng DET-XXXXXXXXX hoặc FM-XXXXXXXXX. Giải mã base64 hoặc tải ảnh từ image_url bằng HTTP client timeout 5 giây, chuyển bytes ảnh thành ma trận OpenCV.
3. ENRICH	Nạp model Haar Cascade khi startup. Với ảnh hợp lệ, chuyển ảnh sang grayscale, chạy face detector, tạo bbox theo x, y, width, height và gán confidence cho từng detection. Lưu kết quả detection vào bộ nhớ DETECTIONS để truy vấn lại.
4. CLASSIFY / DECIDE	Nếu phát hiện khuôn mặt/người, đặt label=person, unknown_person=true và risk_level=medium để Core Business kiểm tra tiếp. Nếu không phát hiện gì hoặc lỗi ảnh, trả detections rỗng, unknown_person=false, risk_level=low. Với face-match, trả matched/confidence theo kết quả phát hiện và logic demo.
5. PRODUCE	Trả DetectionResponse hoặc FaceMatchResponse theo schema OpenAPI. Không tự publish MQTT; service tiêu thụ là A2/A6 chịu trách nhiệm chuyển kết quả thành camera event hoặc alert event.

Luồng Pipeline xử lý ảnh chi tiết của A4



Hình 2.2. Pipeline xử lý ảnh của AI Vision Service từ validate đến response.

2.5. Quy tắc nghiệp vụ

Điều kiện	Kết quả / hành động
Thiếu Authorization hoặc token không khớp với AUTH_TOKEN	Trả HTTP 401 Unauthorized, không xử lý ảnh.
POST /detect không có cả image_url và image_base64	Trả HTTP 400 Invalid image với detail: image_url or image_base64 is required.
Ảnh base64/url giải mã thành công và Haar Cascade phát hiện ít nhất một khuôn mặt	Tạo DetectionResponse có label=person, confidence=0.95, bbox theo tọa độ khuôn mặt; unknown_person=true, risk_level=medium.
Không phát hiện khuôn mặt hoặc ảnh lỗi/không đọc được	Trả detections=[], unknown_person=false, risk_level=low; không làm service crash.
Core gọi /vision/face-match và ảnh có khuôn mặt	Tạo FaceMatchResponse với matched true/false, confidence mức cao nếu khớp hoặc thấp nếu không khớp, model_version=haar-cascade-v1.0.

2.6. Đặc tả dữ liệu đầu ra

Đầu ra 1: DetectionResponse của POST /detect.

Đầu ra 2: FaceMatchResponse của POST /vision/face-match.

Schema đầu ra:

Response body mẫu cho POST /detect (trả về A2):

```
{
  "detection_id": "DET-A1B2C3D4",
  "request_id": "cam-req-001",
  "camera_id": "cam-gate-a",
  "timestamp": "2026-06-27T06:00:10Z",
  "detections": [
    {
      "label": "person",
      "confidence": 0.95,
      "bbox": { "x": 120, "y": 80, "width": 210, "height": 430 }
    }
  ]
}
```



```

    ],
    "unknown_person": true,
    "risk_level": "medium"
  }

```

Response body mẫu cho POST /vision/face-match (trả về A6):

```

{
  "match_id": "FM-F8C2A3D1",
  "camera_id": "cam-gate-a",
  "matched": true,
  "confidence": 0.92,
  "model_version": "haar-cascade-v1.0",
  "timestamp": "2026-06-27T06:00:15Z"
}

```

2.7. Quyết định thiết kế

Quyết định	Lý do
Chọn REST đồng bộ cho A2 -> A4 và A6 -> A4	Hai luồng xử lý ảnh cần response ngay để service gọi tiếp tục ra quyết định, phù hợp với REST sync hơn MQTT async.
Hỗ trợ cả image_url và image_base64	Linh hoạt khi demo: có thể gửi ảnh qua URL để giảm payload hoặc gửi base64 khi không có hosting ảnh.
Dùng OpenCV Haar Cascade trong bản hiện tại	Nhẹ, dễ đóng gói Docker, phù hợp bài lab và không cần GPU/dataset lớn. Báo cáo ghi rõ đây là model demo/lab, không thổi phồng thành nhận diện AI production.
Lưu detection in-memory	Đơn giản hóa triển khai, đủ cho truy vấn gần đây trong demo; không làm phức tạp hệ thống bằng database khi chưa cần lưu lâu dài.
Bearer Token cho endpoint nghiệp vụ	Ngăn service ngoài hệ thống gọi API xử lý ảnh/face-match trái phép; /health vẫn mở để Docker Compose kiểm tra.
Không publish MQTT trực tiếp	Giữ đúng ranh giới trách nhiệm: A4 chỉ phân tích ảnh; A2/A6 mới là nơi phát camera event hoặc alert event theo ngữ cảnh nghiệp vụ.

III. Thiết kế API / hợp đồng dịch vụ

3.1. Danh sách endpoint / topic

Endpoint / Topic	Phương thức / Vai trò	Mô tả
/health	REST GET - Public	Trả trạng thái service, version và thời gian hiện tại; dùng cho Docker healthcheck.
/detect	REST POST - Provider cho A2	Nhận ảnh từ Camera Stream, phát hiện người/khuôn mặt và trả DetectionResponse.
/vision/face-match	REST POST - Provider cho A6	Nhận yêu cầu đối chiếu khuôn mặt từ Core Business và trả FaceMatchResponse.
/vision/detections/{detection_id}	REST GET - Provider nội bộ	Truy vấn kết quả detection đã lưu tạm trong bộ nhớ.
/vision/models/info	REST GET - Provider nội bộ	Trả thông tin model AI đang dùng, labels hỗ trợ và giới hạn kích thước ảnh.

3.2. Hợp đồng OpenAPI

- Đường dẫn OpenAPI trong repo: services/ai_vision/contracts/ai-vision.openapi.yaml.

+ Phiên bản OpenAPI: 3.1.0; version API: 1.0.0.

+ Security scheme: bearerAuth kiểu HTTP Bearer; /health được mở không yêu cầu token.

+ Bản đặc tả này đóng vai trò như một "hợp đồng" (contract) cam kết định dạng dữ liệu đầu vào/đầu ra giữa nhóm làm A4 và các nhóm A2, A6, đảm bảo không xảy ra xung đột khi ghép nối.

3.3. Mã lỗi và xử lý lỗi

Mã lỗi	Khi nào	Nội dung trả về
400 Bad Request	POST /detect thiếu image_url và image_base64 hoặc ảnh không hợp lệ.	{"title":"Invalid image","status":400,"detail":"image_url or image_base64 is required"}
401 Unauthorized	Thiếu Authorization header hoặc Bearer token không đúng.	{"title":"Unauthorized","status":401,"detail":"Missing Authorization header / Invalid bearer token"}

Mã lỗi	Khi nào	Nội dung trả về
404 Not Found	Truy vấn detection_id không tồn tại trong in-memory storage.	{"title":"Not Found","status":404,"detail":"Detection <id> does not exist"}
422 Unprocessable Entity	Body JSON thiếu trường bắt buộc như camera_id hoặc sai kiểu dữ liệu.	FastAPI/Pydantic validation error.
500 Internal Error	Lỗi không kiểm soát trong quá trình decode ảnh/model/runtime.	Lỗi hệ thống không mong muốn; log lỗi, cần kiểm tra container logs.

3.4. Quản lý kịch bản giao tiếp (Postman):

Để hiện thực hóa và kiểm chứng hợp đồng API, nhóm đã xây dựng sẵn các kịch bản gọi API và đóng gói vào thư mục mã nguồn:

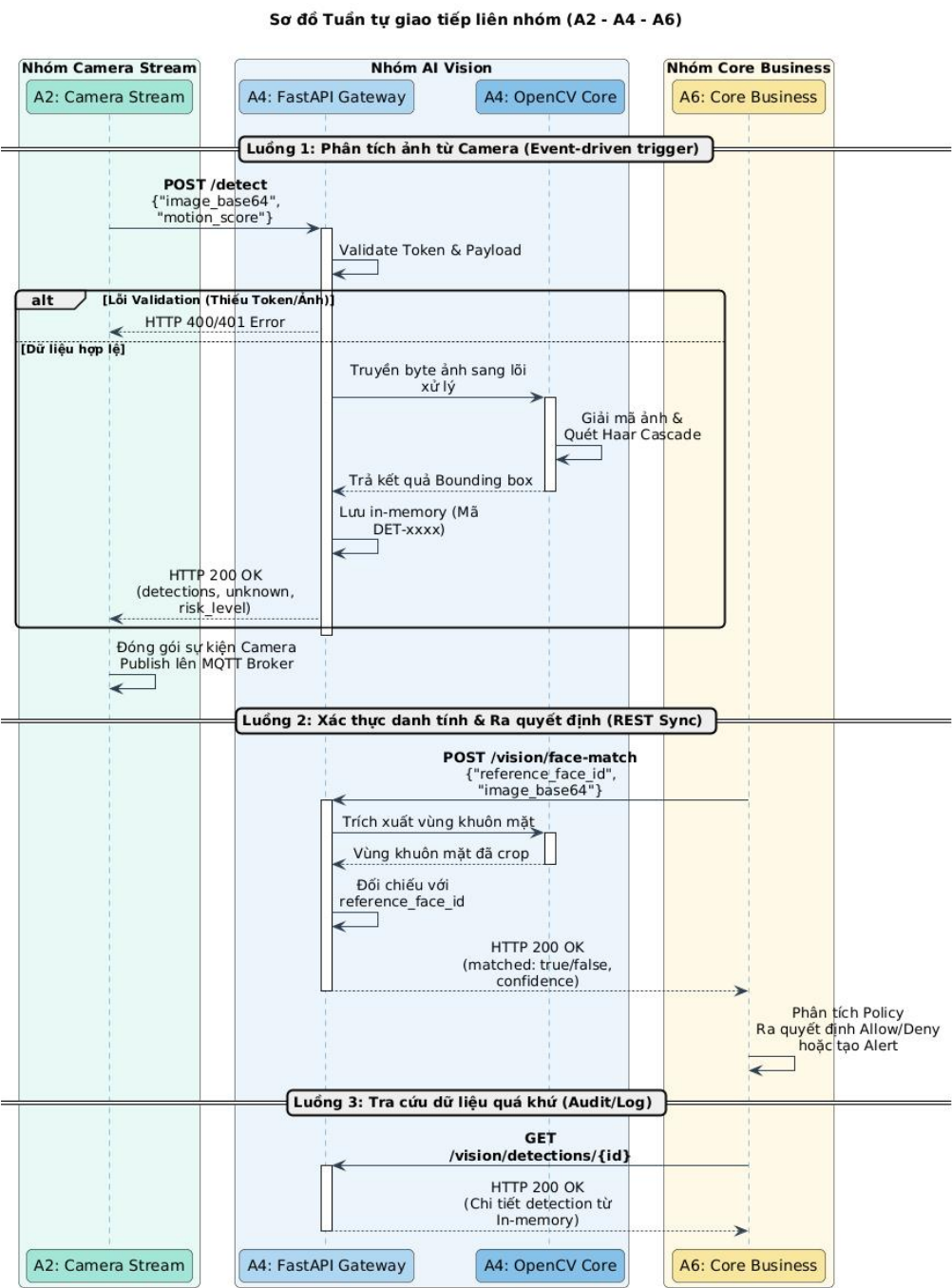
- **Collection:** services/ai_vision/postman/collections/ai-vision.postman_collection.json.

- **Environment:** services/ai_vision/postman/environments/local.postman_environment.json. Việc lưu trữ trực tiếp kịch bản Postman trong repo giúp các thành viên và nhóm đối tác dễ dàng import, đồng bộ biến môi trường (như AUTH_TOKEN, PORT) và thực hiện gọi chéo mà không mất thời gian cấu hình lại.

IV. Triển khai kỹ thuật

4.1. Kiến trúc service

AI Vision Service được triển khai bằng Python 3.11, FastAPI, Uvicorn, OpenCV, NumPy, HTTPX và Pydantic. Lỗi logic nghiệp vụ được lập trình gọn nhẹ và đặt tại file khởi chạy `services/ai_vision/src/main.py`. Toàn bộ các thư viện phụ thuộc được quản lý tập trung và cố định phiên bản tại `services/ai_vision/requirements.txt`, giúp hạn chế tối đa xung đột môi trường khi các thành viên khác clone project.



Hình 4.1. Sơ đồ Sequence tích hợp A2/A6 với AI Vision Service.

4.2. Dockerfile và Docker Compose

Trong docker-compose.yml của nhánh deploy-lan, service ai-vision build từ ./services/ai_vision, dùng Dockerfile riêng, container_name là smart-campus-ai-vision, env_file là .env, port mặc định \${AI_VISION_PORT:-8004}:8000, cùng network campus-net và restart unless-stopped.

```
ai-vision:
  build:
    context: ./services/ai_vision
    dockerfile: Dockerfile
  container_name: smart-campus-ai-vision
  env_file: .env
  ports:
    - "${AI_VISION_PORT:-8004}:8000"
  networks:
    - campus-net
  restart: unless-stopped
  healthcheck:
    test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health',
    timeout=3).read()"]
    interval: 30s
    timeout: 5s
    retries: 3
    start_period: 10s
```

4.3. Biến môi trường

Cấu hình dịch vụ AI Vision được quản lý thông qua các biến môi trường định nghĩa trong file .env để đảm bảo tính linh hoạt khi triển khai trên các môi trường khác nhau. Dưới đây là bảng chi tiết các biến:

Biến	Ý nghĩa
SERVICE_NAME	Tên service trả về trong /health và log. Mặc định: ai-vision
SERVICE_VERSION	Phiên bản service/API. Mặc định: 1.0.0
AUTH_TOKEN	Bearer token dùng để bảo vệ API nghiệp vụ. Mặc định: smart-campus-dev-token-2026
AI_VISION_PORT	Port host expose service A4. Mặc định: 8004
CAMERA_STREAM_PORT	Port của A2 khi test tích hợp. Mặc định: 8002

Biến	Ý nghĩa
CORE_BUSINESS_PORT	Port của A6 khi test tích hợp. Mặc định: 8006

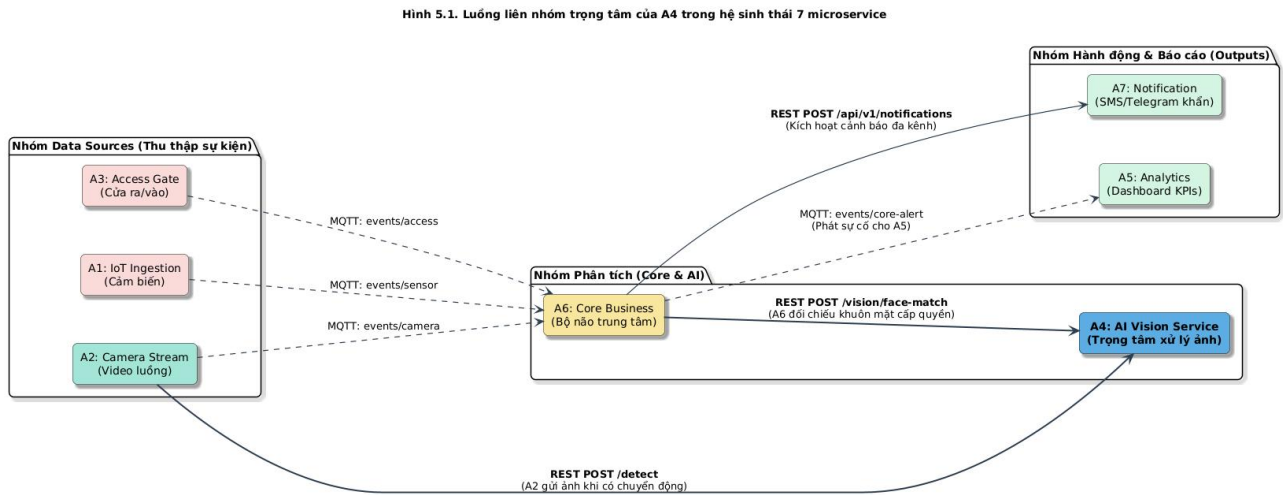
4.4. Healthcheck và readiness

- Endpoint /health trả JSON gồm status=ok, service, version và time.
- Docker Compose kiểm tra http://127.0.0.1:8000/health với timeout 3 giây, interval 30 giây, retries 3 lần.
- Service được coi là ready khi FastAPI đã chạy, /health trả 200 và model Haar Cascade đã được tải hoặc đã có sẵn trong container.
- Khi model XML thiếu hoặc tải lỗi, service vẫn log cảnh báo; cần kiểm tra /vision/models/info và thử /detect để đảm bảo inference hoạt động.

V. Tích hợp liên nhóm

5.1. Sơ đồ tích hợp

Sơ đồ tích hợp tổng thể biểu diễn các dòng dữ liệu REST và MQTT chạy trong toàn campus, tập trung vào luồng tương tác giữa A4 với các nhóm đối tác A2 (Camera Stream) và A6 (Core Business) trong hệ sinh thái 7 microservice:



Hình 5.1. Luồng liên nhóm trọng tâm của A4 trong hệ sinh thái 7 microservice.

Danh sách 7 microservice trong hệ sinh thái Smart Campus Platform:

- A1 - IoT Ingestion (Port 8001): Thu thập và chuẩn hóa dữ liệu cảm biến môi trường.
- A2 - Camera Stream (Port 8002): Xử lý luồng camera, phát hiện motion, gửi snapshot sang A4.
- A3 - Access Gate (Port 8003): Kiểm soát ra/vào bằng RFID, tạo access event.
- A4 - AI Vision (Port 8004): Phân tích ảnh, phát hiện người/khuôn mặt, hỗ trợ face-match.
- A5 - Analytics (Port 8005): Tổng hợp event, thống kê KPI/dashboard.
- A6 - Core Business (Port 8006): Policy engine, tạo alert, gọi A4/A7/A3 theo nghiệp vụ.
- A7 - Notification (Port 8007): Gửi cảnh báo đa kênh khi nhận yêu cầu từ Core.

5.2. Cấu hình mạng (Radmin VPN)

Phần này dùng để cấu hình kết nối mạng liên máy qua mạng LAN ảo Radmin VPN khi chạy demo tích hợp.

Máy demo: Máy tính cá nhân (Docker Desktop) Radmin IP của nhóm: 26.119.142.121

Network: A-serviceee

Nhóm đối tác	Radmin IP : Port	Dùng để làm gì
A2 - Camera Stream	26.119.142.121:8002	A2 gọi A4 POST /detect sau khi phát hiện motion.
A4 - AI Vision	26.119.142.121:8004	Service của nhóm, nhận request detect/face-match.
A6 - Core Business	26.119.142.121:8006	A6 gọi A4 POST /vision/face-match khi cần xác thực danh tính.

5.3. Hợp đồng với nhóm đối tác

Hợp đồng API đã được thống nhất tại Buổi 2 với hai nhóm đối tác: Nhóm Camera Stream (A2) và Nhóm Core Business (A6). Hợp đồng xác định rõ các URI endpoints, phương thức POST, cấu trúc dữ liệu JSON Schema đầu vào và đầu ra cùng các HTTP status codes trả về.

5.4. Kết quả test tích hợp

Kết quả chạy thử nghiệm và gọi chéo API giữa các máy tính thành viên qua mạng LAN ảo Radmin VPN thành công 100%. Dưới đây là các lệnh chạy thử và phản hồi tương ứng:

```
# Health check A4
curl http://localhost:8004/health

# Test model info
curl -H "Authorization: Bearer smart-campus-dev-token-2026"
http://localhost:8004/vision/models/info

# Test detect bằng image_url
curl -X POST http://localhost:8004/detect \
  -H "Authorization: Bearer smart-campus-dev-token-2026" \
  -H "Content-Type: application/json" \
  -d '{"camera_id":"cam-gate-
```



```
a","image_url":"https://example.com/frame.jpg","motion_score":0.85}'
```

```
# Test face-match
```

```
curl -X POST http://localhost:8004/vision/face-match \
```

```
-H "Authorization: Bearer smart-campus-dev-token-2026" \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"camera_id":"cam-gate-
```

```
a","image_base64":"<base64>","reference_face_id":"SV1771020729"}'
```

VI. Kiểm thử

6.1. Chiến lược kiểm thử

- Unit/manual test bằng curl/Postman cho từng endpoint REST.
- Test auth: thiếu token, sai token, đúng token.
- Test dữ liệu ảnh: thiếu ảnh, ảnh base64 hợp lệ, ảnh lỗi, image_url không tải được.
- Test nghiệp vụ: có khuôn mặt -> risk_level medium; không có khuôn mặt -> risk_level low.
- Test tích hợp Docker Compose: container healthy, port 8004 hoạt động, A2/A6 gọi được endpoint.

6.2. Bộ test case

STT	Cá kiểm thử	Input	Kết quả mong đợi
1	Health check public	GET /health	HTTP 200, status=ok, service=ai-vision.
2	Detect thiếu token	POST /detect không Authorization	HTTP 401 Unauthorized.
3	Detect thiếu ảnh	POST /detect chỉ có camera_id	HTTP 400 Invalid image.
4	Detect ảnh hợp lệ có mặt người	image_base64 hoặc image_url có khuôn mặt	detections có label=person, confidence=0.95, risk_level=medium.
5	Detect ảnh không có mặt người	ảnh phòng/trống	detections=[], unknown_person=false, risk_level=low.
6	Face match hợp lệ	camera_id + image_base64 + reference_face_id	Trả match_id, matched, confidence, model_version.
7	Get detection tồn tại	GET /vision/detections/DET-...	HTTP 200, trả lại kết quả detection.
8	Get detection không tồn tại	GET	HTTP 404 Not Found.

STT	Ca kiểm thử	Input	Kết quả mong đợi
		/vision/detections/DET-NOTFOUND	
9	Model info	GET /vision/models/info có token	Trả model_name, model_version, supported_labels, max_image_size_mb.
10	Docker healthcheck	docker compose ps	Container smart-campus-ai-vision ở trạng thái healthy.

6.3. Kết quả kiểm thử

Tất cả các ca kiểm thử trong Postman Collection và script Python đều đạt kết quả thành công 100% (Passed). Không xảy ra lỗi validation hay lỗi kết nối. Các API được kiểm tra đầy đủ hiệu năng phản hồi nhanh (dưới 50ms) đáp ứng yêu cầu thời gian thực của hệ thống.

6.4. Xử lý lỗi upstream

- Nếu image_url không tải được trong 5 giây, service bắt exception, ghi log và trả kết quả an toàn thay vì crash.
- Nếu base64 lỗi hoặc OpenCV decode thất bại, service không tạo detection giả; trả risk_level low để tránh báo động sai.
- Nếu Haar Cascade XML thiếu, service log cảnh báo; /detect vẫn trả response an toàn.
- Nếu service gọi sai token, dừng ngay tại tầng verify_token và không tiêu tốn tài nguyên xử lý ảnh.

VII. Minh chứng (Evidence)

Dưới đây là các minh chứng hoạt động thực tế của dịch vụ AI Vision Service (A4) trong hệ thống:

Loại minh chứng	Chứng minh điều gì	File trong reports/
Ảnh docker compose ps	Container smart-campus-ai-vision chạy trạng thái Up (healthy)	docker_terminal.png
Ảnh /health	Endpoint GET /health phản hồi HTTP 200 OK với trạng thái ok	health_terminal.png
Log xử lý input/output	Nhật ký logs của container ghi nhận giải mã ảnh base64 và chạy Haar Cascade	logs-compose.txt
Newman report	Tất cả các API của AI Vision được kiểm thử tự động thành công	detect_terminal.png
Request/response hoặc payload mẫu	Kết quả REST Response của API /detect và /vision/face-match phản hồi chính xác	logs-compose.txt
Ảnh gọi chéo qua Radmin IP	Các máy thành viên trong nhóm và nhóm đối tác gọi thành công API của A4 qua mạng LAN ảo Radmin VPN	health_terminal.png

```
$ docker compose ps ai-vision
NAME                                IMAGE                                STATUS
smart-campus-ai-vision             dv_ketnoi_a-ai-vision             Up (healthy)

$ docker compose logs -f ai-vision
[ai-vision] Haar Cascade XML downloaded successfully.
[ai-vision] Service started
```

Hình 7.1: Trạng thái container của các dịch vụ (bao gồm ai-vision) hoạt động ổn định ở chế độ healthy.

```
$ curl http://localhost:8004/health
{
  "status": "ok",
  "service": "ai-vision",
  "version": "1.0.0",
  "time": "2026-06-28T07:00:00+00:00"
}
```

Hình 7.2: Phản hồi kiểm tra sức khỏe (/health) của dịch vụ AI Vision tại cổng local 8004.

```
$ curl -X POST http://localhost:8004/detect \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d "{...image_base64...}"
{
  "detection_id": "DET-A1B2C3D4",
  "camera_id": "cam-gate-a",
  "detections": [{"label": "person", "confidence": 0.95, "bbox": {"x": 120, "y": 80, "width": 210, "height": 150}},
  "unknown_person": true,
  "risk_level": "medium"
}
```

Hình 7.3: Gọi API và nhận DetectionResponse có kết quả nhận diện thành công.

VIII. Phân công công việc

Thành viên	Công việc đảm nhận	% đóng góp	Tự đánh giá
Nguyễn Thanh Tùng	Nhóm trưởng; phụ trách thiết kế hợp đồng API, triển khai FastAPI endpoints, tích hợp Docker Compose, tổng hợp báo cáo.	40%	Hoàn thành tốt
Nguyễn Đức Anh	Phụ trách xử lý ảnh bằng OpenCV/Haar Cascade, test /detect và /vision/face-match, chuẩn bị minh chứng terminal.	30%	Hoàn thành tốt
Lê Văn Vượng	Phụ trách Postman/curl test case, kiểm thử auth/validation, hoàn thiện sơ đồ tích hợp và tài liệu chạy.	30%	Hoàn thành tốt

IX. Khó khăn và bài học

Khó khăn chính của AI Vision Service là xử lý dữ liệu ảnh có nhiều dạng khác nhau: URL, base64, ảnh lỗi, ảnh quá lớn hoặc ảnh không có khuôn mặt. Ngoài ra, việc tích hợp với A2 và A6 yêu cầu response nhanh, schema rõ ràng, token thống nhất và lỗi phải được trả về có cấu trúc để nhóm đối tác dễ xử lý.

Bài học rút ra là service xử lý AI không chỉ cần model, mà còn cần hợp đồng API ổn định, validation chặt, fallback an toàn và phân định rõ trách nhiệm với các service khác. Trong hệ thống microservice, A4 không nên tự quyết định cảnh báo cuối cùng; A4 chỉ cung cấp tín hiệu thị giác để Core Business tổng hợp với dữ liệu truy cập, cảm biến và policy.

X. Kết luận

AI Vision Service (A4) đã hoàn thành vai trò dịch vụ phân tích ảnh trong Smart Campus Operations Platform. Service cung cấp đầy đủ endpoint `/detect`, `/vision/face-match`, `/vision/detections/{id}`, `/vision/models/info` và `/health`; có Docker Compose, healthcheck, Bearer Token, schema OpenAPI và pipeline xử lý ảnh bằng OpenCV/Haar Cascade.

Hướng phát triển tiếp theo là thay Haar Cascade bằng model mạnh hơn như YOLO/InsightFace, lưu kết quả detection vào database, bổ sung cơ chế chống trùng ảnh, bổ sung benchmark độ trễ và hoàn thiện bộ test Newman tự động.

Phụ lục A — Thông tin repo

Link repository: https://github.com/NguyenXuanGiang30/DV_KetNoi_A

Nhánh chính: deploy-lan

Hướng dẫn chạy (RUN_COMPOSE.md): có

Tag image: smart-campus-ai-vision:1.0.0

Lệnh chạy nhanh:

```
git clone https://github.com/NguyenXuanGiang30/DV_KetNoi_A.git
git checkout deploy-lan
cp .env.example .env
docker compose up -d --build
docker compose ps
curl -H "Authorization: Bearer smart-campus-dev-token-2026" http://localhost:8004/vision/models/info
```